

INSTITUTO TECNOLÓGICO SUPERIOR DEL OCCIDENTE DEL ESTADO DE HIDALGO

DIVISIÓN DE INGENIERÍA EN SISTEMAS COMPUTACIONALES

Manual Técnico y Documentación de Código Fuente (Versión 1.0 y Versión 2.0)

Estación de Diagnóstico Cardiovascular y Calibración Fotométrica de Piel

Sistema Embebido ESP32 Mini, Acelerometría MPU6050, Cancelación de Movimiento NLMS, Filtrado Butterworth SOS, Monitoreo de Batería Li-Po y Calibración Clínica

Proyecto:

Monitor Continuo No Invasivo de Signos Vitales (BPM, Presión Arterial PWA y SpO2) con Cancelación Adaptativa de Ruido de Movimiento y Compensación Melánica Cutánea

Asesores:

- Mtra. Lorena Mendoza Guzmán
- Mtra. Cristy Elizabeth Aguilar Ojeda

Alumno / Desarrollador:

- Alejandro Pérez

Grupo:

- 8.º Semestre - Grupo A
-

Contenido

- **1. Módulos de Firmware Embebido - Versión 1.0 (Línea Base Histórica)**
 - 1.1 Inicialización de Hardware e Interfaz I2C del Sensor MAX30102 (v1)
 - Código
 - 1.2 Configuración de Registros Ópticos y Parámetros Biofísicos del Sensor (v1)
 - Código
 - 1.3 Servidor BLE GATT, Anuncios y Empaquetado Binario de Telemetría (14 Bytes)
 - Código
 - 1.4 Manejador de Comandos de Control Inalámbrico (START / STOP) (v1)
 - Código
 - 1.5 Bucle Principal de Muestreo y Transmisión a 100 Hz (v1)
 - Código
- **2. Módulos de Firmware Embebido - Versión 2.0 (Producción Autónoma y Sensores)**
 - 2.1 Bus I2C Compartido (400 kHz) y Configuración del Acelerómetro MPU6050 ($\pm 2g$)
 - Código
 - 2.2 Monitoreo de Batería con Divisor Resistivo (ADC1) e Histéresis Clínica
 - Código
 - 2.3 Empaquetamiento y Notificación de la Trama Extendida de 20 Bytes
 - Código
 - 2.4 Característica GATT de Telemetría de Batería y Banderas de Estado
 - Código
 - 2.5 Reconexión BLE Automática y Bucle Asíncrono Completo del Firmware v2
 - Código
- **3. Módulos de la Aplicación de Escritorio (Python / PyQt6) - Versión 2.0**
 - 3.1 Declaración de Dependencias, Librerías del Sistema y Constantes Globales
 - Código
 - 3.2 Filtro Digital IIR Butterworth en Secciones de Segundo Orden (`ButterworthFilter`)
 - Código
 - 3.3 Filtro Adaptativo NLMS para Cancelación de Ruido de Movimiento (`NLMSFilter`)
 - Código
 - 3.4 Desempaquetado Asíncrono de Telemetría de 20 Bytes y Batería (`BLEWorker`)
 - Código
 - 3.5 Inicialización de Búferes Extendidos, Calibración y Estados (`MainWindow.__init__`)

- Código
- 3.6 Construcción del HUD Clínico y Activación por Defecto del Filtro Butterworth
(`MainWindow.setup_ui`)
 - Código
- 3.7 Diálogo de Calibración Clínica Individual con 3 Lecturas de Esfigmomanómetro
(`ClinicalCalibrationDialog`)
 - Código
- 3.8 Clasificación Automática de Tono de Piel mediante Nivel Continuo DCraw
(`MainWindow._update_auto_skin_tone`)
 - Código
- 3.9 Motor Colorimétrico y Análisis Fotográfico de Respaldo por Espacio CIELab e ITA
(`MainWindow.analyze_skin_photo`)
 - Código
- 3.10 Algoritmo de Detección de Latidos sobre Señal Limpia post-NLMS y Mediana RR
(`MainWindow.calculate_bpm`)
 - Código
- 3.11 Modelo Hemodinámico PWA sin Truncamiento Rígido con Calibración Individual
(`MainWindow.estimate_blood_pressure`)
 - Código
- 3.12 Estimación Espectral de SpO2 por Ratio de Ratios con Compensación de Melanina
(`MainWindow.calculate_spo2`)
 - Código
- 3.13 Máquina de Estados de Movimiento, Detección de Contacto y Jerarquía de Alertas
(`MainWindow.update_frequency`)
 - Código
- 3.14 Renderizado Dinámico en Tiempo Real en Matplotlib con Señales NLMS
(`MainWindow.update_plot`)
 - Código
- 3.15 Corrección del Error en Botón de Grabación y Manejo de Búferes Clínicos
(`MainWindow.start_recording`)
 - Código
- 3.16 Exportación CSV Extendida (N=30) con Banderas y Validación Estadística
(`MainWindow.save_csv`)
 - Código
- 3.17 Punto de Entrada Principal con Bucle Asíncrono `qasync` (`main`)
 - Código
- **4. Módulos de la Aplicación Web (HTML5 / CSS3 / ES6+) - Versión 2.0**

- 4.1 Estructura Semántica del Monitor Web y Badges de Telemetría (`landing/index.html`)
 - Código
 - 4.2 Modal de Calibración de Esfigmomanómetro en la Web (`landing/index.html`)
 - Código
 - 4.3 Estilizado Visual Hospitalario, Badges de Energía y Banner de Alerta (`landing/style.css`)
 - Código
 - 4.4 Filtro Digital Butterworth SOS en JavaScript (`landing/index.js`)
 - Código
 - 4.5 Filtro Adaptativo NLMS para Cancelación de Ruido en JavaScript (`landing/index.js`)
 - Código
 - 4.6 Conexión GATT y Servicio Web Bluetooth con Característica de Batería (`landing/index.js`)
 - Código
 - 4.7 Decodificación de Trama de 20 Bytes y Aceleración Física Triaxial (`landing/index.js`)
 - Código
 - 4.8 Motor de Detección Automática de Tono de Piel por Razón DCraw en Web (`landing/index.js`)
 - Código
 - 4.9 Corrección de la Indexación del Array en el Selector de Piel del DOM (`landing/index.js`)
 - Código
 - 4.10 Modelo Hemodinámico PWA Web con Offset Clínico Individual (`landing/index.js`)
 - Código
 - 4.11 Bucle Gráfico de Animación a 60 FPS con Persistencia de Brillo (`landing/index.js`)
 - Código
 - 4.12 Exportador y Generador de Dataset CSV Extendido en el Navegador (`landing/index.js`)
 - Código
 - **5. Módulos de Empaquetado, Sincronización y Validación Clínica**
 - 5.1 Especificación PyInstaller para Generación del Ejecutable Autónomo Windows (`tensiometro-final.spec`)
 - Código
 - 5.2 Automatización de Control de Versiones con Descarte de Datasets Pesados (`push.bat` y `.gitignore`)
 - Código
 - 5.3 Protocolo de Validación Estadística Clínica (RMSE < 5 mmHg, Bland-Altman e ICC > 0.85)
 - Código
-

1. Módulos de Firmware Embebido - Versión 1.0 (Línea Base Histórica)

1.1 Inicialización de Hardware e Interfaz I2C del Sensor MAX30102 (v1)

Configura las líneas de comunicación serie I2C entre el microcontrolador ESP32 y el circuito integrado MAX30102. Inicializa los pines GPIO 21 (SDA) y GPIO 22 (SCL) operando a una frecuencia de reloj de 400 kHz (Fast-Mode). Esto asegura que el búfer interno del sensor no se desborde durante la adquisición continua a 100 Hz.

Hace uso de la librería `Wire.h` integrada en el núcleo de Arduino para ESP32 y la librería de bajo nivel `MAX30105.h`.

Código

```
#include <Wire.h>
#include "MAX30105.h"

MAX30105 particleSensor;

#define I2C_SDA_PIN 21
#define I2C_SCL_PIN 22
#define I2C_SPEED 400000 // 400 kHz Fast Mode

void setupI2C_v1() {
    Serial.begin(115200);
    Serial.println("[INICIALIZACIÓN v1] Configurando bus I2C...");

    Wire.begin(I2C_SDA_PIN, I2C_SCL_PIN, I2C_SPEED);

    if (!particleSensor.begin(Wire, I2C_SPEED_FAST)) {
        Serial.println("[ERROR CRÍTICO] Sensor MAX30102 no responde en dirección 0x57.");
        while (1) {
            delay(1000);
        }
    }
    Serial.println("[OK] Sensor MAX30102 enlazado correctamente al bus I2C.");
}
```

1.2 Configuración de Registros Ópticos y Parámetros Biofísicos del Sensor (v1)

Programa los registros de configuración del MAX30102 para optimizar la toma de fotoplethysmografía en la cara ventral de la muñeca. Se establece una corriente de excitación de 12.0 mA en los emisores ópticos para lograr penetración dérmica adecuada sin saturar los fotodiodos ni generar autocalentamiento. Se desactiva el promediado interno para mantener la resolución temporal de 10 ms (100 Hz), se activa el modo de doble iluminación simultánea (Rojo e Infrarrojo) y se fija el ancho de pulso a 411 microsegundos para obtener una resolución de conversión analógica-digital de 18 bits.

Código

```
void configureMAX30102Parameters_v1() {
    byte ledBrightness = 60; // Nivel de corriente: ~12.0 mA por LED
    byte sampleAverage = 1; // Sin promedio (1 muestra por conversión para capturar muesca díc
ota)
    byte ledMode = 2;        // Modo 2 = Rojo e Infrarrojo simultáneos
    int sampleRate = 100;    // Tasa de muestreo = 100 Hz (resolución temporal = 10 ms)
    int pulseWidth = 411;    // Ancho de pulso = 411 us (ADC a 18 bits de resolución)
    int adcRange = 4096;     // Fondo de escala fotométrico = 4096 nA (rango dinámico óptimo)

    particleSensor.setup(ledBrightness, sampleAverage, ledMode, sampleRate, pulseWidth, adcRang
e);
    particleSensor.setPulseAmplitudeRed(ledBrightness);
    particleSensor.setPulseAmplitudeIR(ledBrightness);
    particleSensor.clearFIFO();
    Serial.println("[OK] Registros del MAX30102 configurados exitosamente (v1).");
}
```

1.3 Servidor BLE GATT, Anuncios y Empaquetado Binario de Telemetría (14 Bytes)

Implementa el servidor Bluetooth Low Energy (BLE) bajo la arquitectura de perfiles de atributos genéricos (GATT). Define el servicio de diagnóstico médico con su UUID estandarizado y crea la característica de notificación de datos. Para evitar la saturación de la pila de radiofrecuencia, el empaquetador condensa 2 muestras consecutivas de 18 bits de ambos canales (Rojo e Infrarrojo) codificadas en enteros sin signo de 24 bits Big-Endian, junto a un contador incremental de secuencia de 16 bits en Little-Endian, conformando una trama fija de 14 bytes enviada a 50 Hz (100 muestras/segundo reales).

Código

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

#define SERVICE_UUID          "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define DATA_CHAR_UUID       "beb5483e-36e1-4688-b7f5-ea07361b26a8"

BLEServer* pServer = NULL;
BLECharacteristic* pDataChar = NULL;
bool deviceConnected = false;
uint16_t packetSequence = 0;

void notifyTelemetry14Bytes(uint32_t r1, uint32_t ir1, uint32_t r2, uint32_t ir2) {
    if (!deviceConnected) return;

    uint8_t payload[14];

    // Muestra 1: Canal Rojo (24 bits Big-Endian)
    payload[0] = (r1 >> 16) & 0xFF;
    payload[1] = (r1 >> 8) & 0xFF;
    payload[2] = r1 & 0xFF;

    // Muestra 1: Canal Infrarrojo (24 bits Big-Endian)
    payload[3] = (ir1 >> 16) & 0xFF;
    payload[4] = (ir1 >> 8) & 0xFF;
    payload[5] = ir1 & 0xFF;

    // Muestra 2: Canal Rojo (24 bits Big-Endian)
    payload[6] = (r2 >> 16) & 0xFF;
    payload[7] = (r2 >> 8) & 0xFF;
```

```
payload[8] = r2 & 0xFF;

// Muestra 2: Canal Infrarrojo (24 bits Big-Endian)
payload[9] = (ir2 >> 16) & 0xFF;
payload[10] = (ir2 >> 8) & 0xFF;
payload[11] = ir2 & 0xFF;

// Contador de Secuencia (16 bits Little-Endian)
payload[12] = packetSequence & 0xFF;
payload[13] = (packetSequence >> 8) & 0xFF;

pDataChar->setValue(payload, 14);
pDataChar->notify();
packetSequence++;
}
```

1.4 Manejador de Comandos de Control Inalámbrico (START / STOP) (v1)

Implementa el receptor asíncrono de comandos de control enviados desde la computadora hacia la característica GATT de escritura (**12345678-1234-1234-1234-123456789abc**). Administra el estado de la máquina de adquisición y resetea contadores para asegurar sincronización de fases temporales.

Código

```
bool streamingActive = false;
uint8_t bufferIndex = 0;

class ControlCallbacksV1: public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
        String rxValue = pCharacteristic->getValue().c_str();
        if (rxValue.length() > 0) {
            if (rxValue == "START") {
                streamingActive = true;
                packetSequence = 0;
                bufferIndex = 0;
                Serial.println("[BLE v1] Comando START recibido. Telemetría activa.");
            } else if (rxValue == "STOP") {
                streamingActive = false;
                Serial.println("[BLE v1] Comando STOP recibido. Suspendiendo transmisiones.");
            }
        }
    }
};
```

1.5 Bucle Principal de Muestreo y Transmisión a 100 Hz (v1)

Gestiona la extracción del búfer FIFO del sensor MAX30102 con temporización estricta de 10 ms (100 Hz). Empaqueta los datos en pares consecutivos de muestras dobles y los envía hacia el cliente receptor.

Código

```
#define SAMPLE_PAIRS_PER_PACKET 2
uint32_t redBuffer[SAMPLE_PAIRS_PER_PACKET];
uint32_t irBuffer[SAMPLE_PAIRS_PER_PACKET];

void loop_v1() {
    if (deviceConnected && streamingActive) {
        particleSensor.check();

        while (particleSensor.available()) {
            redBuffer[bufferIndex] = particleSensor.getRed();
            irBuffer[bufferIndex] = particleSensor.getIR();
            particleSensor.nextSample();
            bufferIndex++;

            if (bufferIndex >= SAMPLE_PAIRS_PER_PACKET) {
                notifyTelemetry14Bytes(redBuffer[0], irBuffer[0], redBuffer[1], irBuffer[1]);
                bufferIndex = 0;
            }
        }
        delay(1);
    }
}
```

2. Módulos de Firmware Embebido - Versión 2.0

(Producción Autónoma y Sensores)

2.1 Bus I2C Compartido (400 kHz) y Configuración del Acelerómetro MPU6050 ($\pm 2g$)

Este componente configura el bus I2C en modo Fast (400 kHz) conectando simultáneamente el sensor óptico MAX30102 en la dirección `0x57` y el acelerómetro inercial MPU6050 en la dirección `0x68`. Programa los registros de control del MPU6050 para despertar el oscilador interno (registro `0x6B`) y fijar el rango dinámico de aceleración a $\pm 2g$ (registro `0x1C`), con una escala de 16384 LSB/g .

Hace uso de la librería de bajo nivel `Wire.h` sin sobrecargas de librerías externas pesadas.

Código

```
#include <Wire.h>

#define MPU6050_ADDR      0x68
#define MPU_PWR_MGMT_1    0x6B
#define MPU_ACCEL_CONFIG  0x1C
#define MPU_ACCEL_XOUT_H  0x3B

int16_t accelX = 0;
int16_t accelY = 0;
int16_t accelZ = 16384; // 1g estático en reposo

bool initMPU6050() {
    Wire.beginTransmission(MPU6050_ADDR);
    Wire.write(MPU_PWR_MGMT_1);
    Wire.write(0x00); // Salir de modo suspensión (despertar oscilador)
    if (Wire.endTransmission() != 0) {
        return false;
    }

    Wire.beginTransmission(MPU6050_ADDR);
    Wire.write(MPU_ACCEL_CONFIG);
    Wire.write(0x00); // Rango de escala completa a  $\pm 2g$ 
    return (Wire.endTransmission() == 0);
}

void readMPU6050Accel() {
    Wire.beginTransmission(MPU6050_ADDR);
```

```
Wire.write(MPU_ACCEL_XOUT_H);  
Wire.endTransmission(false);  
Wire.requestFrom((uint8_t)MPU6050_ADDR, (size_t)6, true);  
  
if (Wire.available() >= 6) {  
    accelX = (Wire.read() << 8) | Wire.read();  
    accelY = (Wire.read() << 8) | Wire.read();  
    accelZ = (Wire.read() << 8) | Wire.read();  
}  
}
```

2.2 Monitoreo de Batería con Divisor Resistivo (ADC1) e Histéresis Clínica

Mide la tensión de la batería Li-Po de 3.7V mediante un divisor simétrico ($100\text{ k}\Omega + 100\text{ k}\Omega$, factor 2.0) conectado al pin GPIO 34 (ADC1). Aplica un promediado digital de 8 muestras para atenuar ruido térmico, deduce la tensión real en milivoltios y el porcentaje de carga remanente. Implementa una lógica clínica de histéresis: si la tensión desciende de 3.40V, activa la bandera de alerta para suspender el cálculo de presión arterial; la alerta únicamente se apaga si la batería se recarga y supera los 3.50V.

Código

```
#define BATTERY_ADC_PIN    34
#define ADC_REF_VOLTAGE    3.3
#define ADC_RESOLUTION     4095.0
#define V_DIVIDER_RATIO    2.0
#define V_BAT_CRITICAL     3.40
#define V_BAT_HYSTERESIS   3.50

uint8_t batteryPct = 100;
uint16_t batteryMv = 4200;
bool lowBatteryActive = false;

void checkBatteryStatus() {
    uint32_t adcSum = 0;
    for (int k = 0; k < 8; k++) {
        adcSum += analogRead(BATTERY_ADC_PIN);
        delayMicroseconds(50);
    }
    float rawAdc = (float)adcSum / 8.0f;
    float voltage = (rawAdc / ADC_RESOLUTION) * ADC_REF_VOLTAGE * V_DIVIDER_RATIO;
    batteryMv = (uint16_t)(voltage * 1000.0f);

    int pct = (int)(((voltage - V_BAT_CRITICAL) / (4.20f - V_BAT_CRITICAL)) * 100.0f);
    batteryPct = (uint8_t)constrain(pct, 0, 100);

    // Histéresis clínica de seguridad
    if (voltage < V_BAT_CRITICAL) {
        lowBatteryActive = true;
    } else if (voltage >= V_BAT_HYSTERESIS) {
        lowBatteryActive = false;
    }
}
```

2.3 Empaquetamiento y Notificación de la Trama Extendida de 20 Bytes

Condensa en una única trama binaria fija de 20 bytes las 2 muestras de 18 bits de ambos canales ópticos (Rojo e Infrarrojo), el contador de secuencia de 16 bits y las 3 componentes de aceleración física triaxial (\$A_x\$, \$A_y\$, \$A_z\$). Se emite por notificación BLE a 50 Hz, garantizando la entrega síncrona de 100 muestras reales por segundo sin pérdidas de paquetes.

Código

```
void notifyTelemetry20Bytes(uint32_t r1, uint32_t ir1, uint32_t r2, uint32_t ir2) {
    if (!deviceConnected) return;

    readMPU6050Accel(); // Actualizar aceleración física

    uint8_t payload[20];

    // Muestra 1: Canal Rojo (24 bits Big-Endian)
    payload[0] = (r1 >> 16) & 0xFF;
    payload[1] = (r1 >> 8) & 0xFF;
    payload[2] = r1 & 0xFF;

    // Muestra 1: Canal Infrarrojo (24 bits Big-Endian)
    payload[3] = (ir1 >> 16) & 0xFF;
    payload[4] = (ir1 >> 8) & 0xFF;
    payload[5] = ir1 & 0xFF;

    // Muestra 2: Canal Rojo (24 bits Big-Endian)
    payload[6] = (r2 >> 16) & 0xFF;
    payload[7] = (r2 >> 8) & 0xFF;
    payload[8] = r2 & 0xFF;

    // Muestra 2: Canal Infrarrojo (24 bits Big-Endian)
    payload[9] = (ir2 >> 16) & 0xFF;
    payload[10] = (ir2 >> 8) & 0xFF;
    payload[11] = ir2 & 0xFF;

    // Contador de Secuencia (16 bits Little-Endian)
    payload[12] = packetSequence & 0xFF;
    payload[13] = (packetSequence >> 8) & 0xFF;

    // Aceleración Triaxial MPU6050 (int16 Little-Endian)
    payload[14] = accelX & 0xFF;
    payload[15] = (accelX >> 8) & 0xFF;
    payload[16] = accelY & 0xFF;
```

```
payload[17] = (accelY >> 8) & 0xFF;  
payload[18] = accelZ & 0xFF;  
payload[19] = (accelZ >> 8) & 0xFF;  
  
pDataChar->setValue(payload, 20);  
pDataChar->notify();  
packetSequence++;  
}
```

2.4 Característica GATT de Telemetría de Batería y Banderas de Estado

Define e implementa la característica BLE dedicada para la telemetría del sistema de energía bajo el UUID estándar `00002a19-0000-1000-8000-00805f9b34fb`. Transmite periódicamente un payload de 4 bytes conteniendo el nivel porcentual (0–100%), la tensión en milivoltios (uint16) y un byte de banderas de estado donde el bit 1 indica la presencia de la alerta activa por batería baja.

Código

```
#define BATTERY_CHAR_UUID    "00002a19-0000-1000-8000-00805f9b34fb"
BLECharacteristic* pBatteryChar = NULL;

void notifyBatteryTelemetry() {
    if (!deviceConnected || pBatteryChar == NULL) return;

    uint8_t flags = 0;
    if (deviceConnected)    flags |= 0x01; // Bit 0: Enlace activo
    if (lowBatteryActive)    flags |= 0x02; // Bit 1: Batería baja activa (<3.4V)

    uint8_t batPayload[4];
    batPayload[0] = batteryPct;
    batPayload[1] = batteryMv & 0xFF;
    batPayload[2] = (batteryMv >> 8) & 0xFF;
    batPayload[3] = flags;

    pBatteryChar->setValue(batPayload, 4);
    pBatteryChar->notify();
}
```

2.5 Reconexión BLE Automática y Bucle Asíncrono Completo del Firmware v2

Implementa el callback de desconexión reactivo en el ESP32 Mini: ante la interrupción súbita del enlace inalámbrico, el servidor GATT reinicia inmediatamente sus paquetes de anuncio para permitir una re-vinculación instantánea del software sin requerir reinicio por hardware. En el bucle principal se ejecutan concurrentemente la lectura óptica de 100 Hz y el monitoreo térmico de la batería cada 2 segundos.

Código

```
class ServerCallbacksV2: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
        deviceConnected = true;
        Serial.println("[BLE v2] Enlace establecido con el cliente receptor.");
    }
    void onDisconnect(BLEServer* pServer) {
        deviceConnected = false;
        streamingActive = false;
        packetSequence = 0;
        bufferIndex = 0;
        Serial.println("[BLE v2] Enlace interrumpido. Reanudando transmisiones de anuncio...");
        pServer->startAdvertising(); // Reconexión automática inmediata
    }
};

unsigned long lastBatteryCheck = 0;

void loop_v2() {
    if (millis() - lastBatteryCheck >= 2000) {
        lastBatteryCheck = millis();
        checkBatteryStatus();
        notifyBatteryTelemetry();
    }

    if (deviceConnected && streamingActive) {
        particleSensor.check();

        while (particleSensor.available()) {
            redBuffer[bufferIndex] = particleSensor.getRed();
            irBuffer[bufferIndex] = particleSensor.getIR();
            particleSensor.nextSample();
            bufferIndex++;
        }
    }
}
```

```
        if (bufferIndex >= SAMPLE_PAIRS_PER_PACKET) {  
            notifyTelemetry20Bytes(redBuffer[0], irBuffer[0], redBuffer[1], irBuffer[1]);  
            bufferIndex = 0;  
        }  
    }  
    }  
    delay(1);  
}
```

3. Módulos de la Aplicación de Escritorio (Python / PyQt6) - Versión 2.0

3.1 Declaración de Dependencias, Librerías del Sistema y Constantes Globales

Declara el entorno tecnológico de la estación médica de escritorio, integrando librerías para comunicación asíncrona (`bleak`, `qasync`), álgebra computacional (`numpy`), visualización científica (`matplotlib`) e interfaz de usuario moderna (`PyQt6`).

Código

```
// Equivalente en Python (tensiometro_reconstructed.py)
```

```
import sys
import asyncio
import time
import csv
import math
import numpy as np
from collections import deque
from threading import Lock
from bleak import BleakClient, BleakScanner
from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QPushButton,
                             QLabel, QComboBox, QVBoxLayout, QHBoxLayout,
                             QMessageBox, QFileDialog, QSpinBox, QCheckBox,
                             QProgressBar, QGroupBox, QGridLayout, QFrame,
                             QDialog, QLineEdit, QDoubleSpinBox)
from PyQt6.QtCore import Qt, QTimer, QThread, pyqtSignal
from PyQt6.QtGui import QFont, QImage
from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as FigureCanvas
from matplotlib.figure import Figure
from qasync import QEventLoop

SERVICE_UUID = "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
CHAR_UUID = "beb5483e-36e1-4688-b7f5-ea07361b26a8"
CONTROL_UUID = "12345678-1234-1234-1234-123456789abc"
BATTERY_CHAR_UUID = "00002a19-0000-1000-8000-00805f9b34fb"
EXPECTED_FREQ = 100
DT_SAMPLE = 1.0 / EXPECTED_FREQ
```

3.2 Filtro Digital IIR Butterworth en Secciones de Segundo Orden

(ButterworthFilter)

Implementa un filtro pasabanda IIR Butterworth de orden 4 con frecuencias de corte en 0.5 Hz y 8.0 Hz ($f_s = 100 \text{ Hz}$). Se formula mediante Secciones de Segundo Orden (SOS / Biquads) en Direct Form II para maximizar la estabilidad numérica frente a desbordamientos de coma flotante.

Código

```
class ButterworthFilter:
    """Filtro IIR Butterworth pasabanda de orden 4 (0.5 - 8.0 Hz a fs=100Hz) en cascada SOS."""
    def __init__(self):
        self.sos = [
            [1.78260999e-03, 3.56521998e-03, 1.78260999e-03, -1.29176642e+00, 4.35717576e-01],
            [1.00000000e+00, 2.00000000e+00, 1.00000000e+00, -1.51260719e+00, 7.19524086e-01],
            [1.00000000e+00, -2.00000000e+00, 1.00000000e+00, -1.93751823e+00, 9.38713070e-01],
            [1.00000000e+00, -2.00000000e+00, 1.00000000e+00, -1.97736736e+00, 9.78372608e-01]
        ]
        self.lock = Lock()
        self.reset()

    def filter(self, x):
        with self.lock:
            val = x
            for i in range(4):
                b0, b1, b2, a1, a2 = self.sos[i]
                w1, w2 = self.states[i]
                w = val - a1 * w1 - a2 * w2
                y = b0 * w + b1 * w1 + b2 * w2
                self.states[i] = [w, w1]
                val = y
            return val

    def reset(self):
        with self.lock:
            self.states = [[0.0, 0.0] for _ in range(4)]
```

3.3 Filtro Adaptativo NLMS para Cancelación de Ruido de Movimiento

(NLMSFilter)

Implementa el algoritmo adaptativo de mínimos cuadrados normalizados (NLMS). Toma como entrada deseada la señal PPG filtrada y como señal de referencia la magnitud de aceleración dinámica calculada por el MPU6050. Estima en tiempo real el componente inercial acoplado y lo sustrae mediante un vector de pesos de 24 taps con factor de adaptación $\mu = 0.02$.

Código

```
class NLMSFilter:
    """Filtro Adaptativo NLMS para cancelación de ruido de movimiento mecánico."""
    def __init__(self, num_taps=24, mu=0.02, epsilon=1e-5):
        self.num_taps = num_taps
        self.mu = mu
        self.epsilon = epsilon
        self.weights = np.zeros(num_taps)
        self.buffer = np.zeros(num_taps)
        self.lock = Lock()

    def filter(self, desired, noise_ref, adapt=True):
        with self.lock:
            self.buffer[1:] = self.buffer[:-1]
            self.buffer[0] = noise_ref
            noise_est = float(np.dot(self.weights, self.buffer))
            clean_signal = desired - noise_est

            if adapt:
                power = float(np.dot(self.buffer, self.buffer)) + self.epsilon
                step = (self.mu / power) * clean_signal
                self.weights += step * self.buffer

            return clean_signal

    def reset(self):
        with self.lock:
            self.weights = np.zeros(self.num_taps)
            self.buffer = np.zeros(self.num_taps)
```

3.4 Desempaquetado Asíncrono de Telemetría de 20 Bytes y Batería

(BLEWorker)

Subproceso desacoplado (`QThread`) que gestiona el cliente `Bleak` de Bluetooth Low Energy. Decodifica tramas heredadas de 14 bytes y tramas extendidas de 20 bytes (extrayendo \$A_x, A_y, A_z\$ en formato \$g\$), escucha las notificaciones de la característica de batería y reintenta el enlace automáticamente en caso de pérdida de señal.

Código

```
class BLEWorker(QThread):
    data_received = pyqtSignal(list, list, int, list)
    battery_received = pyqtSignal(int, int, int)
    connection_status = pyqtSignal(bool, str)
    log_message = pyqtSignal(str, str)
    loop_ready = pyqtSignal()

    def _notification_handler(self, sender, data):
        length = len(data)
        if length in (14, 20):
            red_samples = []
            ir_samples = []
            for i in range(2):
                offset = i * 6
                r_val = (data[offset] << 16) | (data[offset+1] << 8) | data[offset+2]
                ir_val = (data[offset+3] << 16) | (data[offset+4] << 8) | data[offset+5]
                red_samples.append(r_val)
                ir_samples.append(ir_val)
            seq = (data[13] << 8) | data[12]

            if length == 20:
                ax = int.from_bytes(data[14:16], byteorder='little', signed=True) / 16384.0
                ay = int.from_bytes(data[16:18], byteorder='little', signed=True) / 16384.0
                az = int.from_bytes(data[18:20], byteorder='little', signed=True) / 16384.0
            else:
                ax, ay, az = 0.0, 0.0, 1.0

            self.data_received.emit(red_samples, ir_samples, seq, [ax, ay, az])
```

3.5 Inicialización de Búferes Extendidos, Calibración y Estados

(`MainWindow.__init__`)

Inicializa los búferes dinámicos de visualización, las estructuras de grabación extendida para 30 variables clínicas, las instancias de los filtros digitales (Butterworth y NLMS) y los parámetros hemodinámicos individuales del sujeto.

Código

```
def __init__(self):
    super().__init__()
    self.setWindowTitle("ESTACIÓN DE MONITOREO CARDIACO - GOBIERNO DE MÉXICO (v2.0)")
    self.resize(1450, 930)

    # Activación por defecto de filtros según requerimiento
    self.dc_filter_enabled = True
    self.filter_red = ButterworthFilter()
    self.filter_ir = ButterworthFilter()
    self.filter_accel = ButterworthFilter()
    self.nlms_red = NLMSFilter(num_taps=24, mu=0.02)
    self.nlms_ir = NLMSFilter(num_taps=24, mu=0.02)

    self.battery_pct = 100
    self.battery_mv = 4200
    self.battery_low = False
    self.motion_state = "REPOSO"

    self.subject_id = "SUJETO-01"
    self.sbp_reference = 120.0
    self.dbp_reference = 80.0
    self.is_clinically_calibrated = False
    self.calib_sbp_offset = 0.0
    self.calib_dbp_offset = 0.0

    self.time_buffer = deque(maxlen=800)
    self.filtered_ir_buffer = deque(maxlen=800)
    self.nlms_ir_buffer = deque(maxlen=800)
```

3.6 Construcción del HUD Clínico y Activación por Defecto del Filtro Butterworth (`MainWindow.setup_ui`)

Construye la interfaz gráfica en estilo de consola hospitalaria institucional. Aplica paleta cromática reglamentaria (Beige `#D4C19C`, Blanco `#FFFFFF`, Guinda `#621132` y Dorado `#B38E5D`). Integra el indicador de batería con nivel porcentual y voltaje, el osciloscopio en Matplotlib y el checkbox del filtro Butterworth **activado por defecto** (`self.filter_checkbox.setChecked(True)`).

Código

```
def setup_ui(self):
    central = QWidget()
    self.setCentralWidget(central)
    main_layout = QVBoxLayout(central)

    # Encabezado HUD con telemetría de energía
    header = QWidget()
    header_layout = QHBoxLayout(header)
    self.battery_label = QLabel(" 🔋 BATERÍA: --% (0.00 V)")
    self.status_label = QLabel(" ⬤ SENSOR: DESCONECTADO")
    header_layout.addWidget(self.battery_label)
    header_layout.addWidget(self.status_label)
    main_layout.addWidget(header)

    # Checkbox de filtrado activo por defecto
    self.filter_checkbox = QCheckBox("Filtro Butterworth (0.5-8Hz)")
    self.filter_checkbox.setChecked(True) # Requerimiento técnico cumplido
    self.nlms_checkbox = QCheckBox("Cancelación NLMS de Movimiento")
    self.nlms_checkbox.setChecked(True)
```

3.7 Diálogo de Calibración Clínica Individual con 3 Lecturas de Esfigmomanómetro (`ClinicalCalibrationDialog`)

Permite ingresar 3 mediciones previas de presión sistólica y diastólica tomadas con esfigmomanómetro clínico oscilométrico de manguito. Promedia las lecturas y deduce los factores de compensación individuales (Δ SBP, Δ DBP), eliminando el truncamiento rígido artificial.

Código

```
class ClinicalCalibrationDialog(QDialog):
    def __init__(self, parent=None, subject_id="SUJETO-01", sbp_ref=120.0, dbp_ref=80.0):
        super().__init__(parent)
        self.setWindowTitle("Calibración Clínica Individual (Esfigmomanómetro)")
        layout = QVBoxLayout(self)

        form_grid = QGridLayout()
        self.txt_id = QLineEdit(subject_id)
        self.sbp1 = QDoubleSpinBox(); self.sbp1.setValue(sbp_ref)
        self.dbp1 = QDoubleSpinBox(); self.dbp1.setValue(dbp_ref)
        self.sbp2 = QDoubleSpinBox(); self.sbp2.setValue(sbp_ref)
        self.dbp2 = QDoubleSpinBox(); self.dbp2.setValue(dbp_ref)
        self.sbp3 = QDoubleSpinBox(); self.sbp3.setValue(sbp_ref)
        self.dbp3 = QDoubleSpinBox(); self.dbp3.setValue(dbp_ref)

        form_grid.addWidget(self.txt_id, 0, 1)
        form_grid.addWidget(self.sbp1, 1, 1); form_grid.addWidget(self.dbp1, 1, 2)
        form_grid.addWidget(self.sbp2, 2, 1); form_grid.addWidget(self.dbp2, 2, 2)
        form_grid.addWidget(self.sbp3, 3, 1); form_grid.addWidget(self.dbp3, 3, 2)
        layout.addLayout(form_grid)

    def get_calibration(self):
        sbp_avg = (self.sbp1.value() + self.sbp2.value() + self.sbp3.value()) / 3.0
        dbp_avg = (self.dbp1.value() + self.dbp2.value() + self.dbp3.value()) / 3.0
        return self.txt_id.text().strip(), sbp_avg, dbp_avg
```

3.8 Clasificación Automática de Tono de Piel mediante Nivel Continuo

DCraw (`MainWindow._update_auto_skin_tone`)

Evalúa el cociente $\text{Ratio}_{\{\text{DC}\}} = \text{DC}_{\{\text{Rojo}\}} / \text{DC}_{\{\text{IR}\}}$ tras 500 muestras de contacto estable en reposo. Clasifica cuantitativamente el fototipo de piel sin necesidad de fotografías y ajusta dinámicamente los factores de corrección de melanina.

Código

```
def _update_auto_skin_tone(self):
    if self.dc_ir_avg <= 0: return
    dc_ratio = self.dc_red_avg / self.dc_ir_avg

    if dc_ratio > 1.25: cat_idx = 0 # Muy clara
    elif dc_ratio > 1.10: cat_idx = 1 # Clara
    elif dc_ratio > 0.95: cat_idx = 2 # Intermedia
    elif dc_ratio > 0.80: cat_idx = 3 # Morena
    elif dc_ratio > 0.65: cat_idx = 4 # Oscura
    else: cat_idx = 5 # Muy oscura

    if self.skin_combo.currentIndex() != cat_idx:
        self.skin_combo.setCurrentIndex(cat_idx)
        name = self.skin_calibration_data[cat_idx]["name"]
        self.log_message(f"Tono de piel adaptado automáticamente por DCraw: {name} (Ratio={dc_ratio:.2f})", "INFO")
```

3.9 Motor Colorimétrico y Análisis Fotográfico de Respaldo por Espacio CIELab e ITA (`MainWindow.analyze_skin_photo`)

Procesa fotografías locales de la piel como módulo de respaldo. Extrae la ROI central (100×100), aplica linealización gamma sRGB, transformación al espacio CIE 1931 XYZ, normalización frente al iluminante estándar D65 y deduce el Ángulo de Tipología Individual (ITA°).

Código

```
def analyze_skin_photo(self):
    # Conversión matemática completa sRGB -> CIELab -> ITA
    ita = math.atan((l_star - 50) / b_star) * (180 / math.pi)
    if ita > 55: idx = 0
    elif ita > 41: idx = 1
    elif ita > 28: idx = 2
    elif ita > 10: idx = 3
    elif ita > -30: idx = 4
    else: idx = 5
    self.skin_combo.setCurrentIndex(idx)
    self.auto_skin_checkbox.setChecked(False)
```

3.10 Algoritmo de Detección de Latidos sobre Señal Limpia post-NLMS y Mediana RR (`MainWindow.calculate_bpm`)

Analiza la señal infrarroja limpia post-NLMS. Suaviza la curva con ventana móvil de 200 ms, detecta los picos sistólicos mediante umbral adaptativo (50 unidades ADC) y aplica una ventana refractaria fisiológica de 400 ms. Calcula la frecuencia cardíaca instantánea a partir de la **mediana** de los intervalos RR válidos ($0.33\text{ s} - 1.5\text{ s}$).

Código

```
def calculate_bpm(self):
    if len(self.nlms_ir_buffer) < 300: return None
    y = np.array(self.nlms_ir_buffer)
    actual_freq = self.actual_freq if self.actual_freq > 10 else EXPECTED_FREQ
    window_size = max(5, int(actual_freq * 0.20)) | 1
    smoothed = np.convolve(y, np.ones(window_size)/window_size, mode='same')

    min_dist = int(actual_freq * 0.40)
    threshold = np.min(smoothed) + (np.max(smoothed) - np.min(smoothed)) * 0.50

    peaks = []
    last_p = -min_dist
    for i in range(1, len(smoothed) - 1):
        if smoothed[i] > smoothed[i-1] and smoothed[i] > smoothed[i+1]:
            if smoothed[i] > threshold and (i - last_p) >= min_dist:
                peaks.append(i); last_p = i

    if len(peaks) < 3: return None
    intervals = np.diff(np.array(self.time_buffer)[peaks])
    valid = intervals[(intervals >= 0.33) & (intervals <= 1.5)]
    if len(valid) < 2: return None
    return 60.0 / np.median(valid)
```

3.11 Modelo Hemodinámico PWA sin Truncamiento Rígido con Calibración Individual (`MainWindow.estimate_blood_pressure`)

Extrae los valles diastólicos mínimos y calcula los tiempos de ascenso (T_s) y descenso (T_d). Se eliminan los truncamientos rígidos arbitrarios ($95-145$ / $60-95$ mmHg) e incorpora el offset clínico deducido de las 3 lecturas previas de esfigmomanómetro.

Código

```
def estimate_blood_pressure(self, bpm, peaks, smoothed):
    # Cálculo de tiempos promedio de subida (Ts) y bajada (Td)
    cal = self.skin_calibration_data[self.skin_combo.currentIndex()]
    base_sbp = 120.0 + 0.15 * (bpm - 70.0) - 75.0 * (avg_rise_sec - 0.12) + cal["sbp_offset"]
    base_dbp = 80.0 + 0.08 * (bpm - 70.0) - 25.0 * (avg_fall_sec - 0.35) + cal["dbp_offset"]

    # Inyección de calibración individual de esfigmomanómetro
    if self.is_clinically_calibrated:
        sbp = base_sbp + self.calib_sbp_offset
        dbp = base_dbp + self.calib_dbp_offset
    else:
        sbp, dbp = base_sbp, base_dbp

    if sbp <= dbp + 20: sbp = dbp + 25
    return int(round(sbp)), int(round(dbp))
```

3.12 Estimación Espectral de SpO2 por Ratio de Ratios con Compensación de Melanina (`MainWindow.calculate_spo2`)

Calcula las componentes de corriente continua (DC) y corriente alterna (AC) en ventanas deslizantes de 3 segundos (300 muestras), calcula el cociente $R = (AC/DC)_{\text{Rojo}} / (AC/DC)_{\text{Infrarrojo}}$ y aplica la compensación melánica.

Código

```
def calculate_spo2(self):
    if len(self.raw_red_buffer) < 300: return None
    r = (ac_red / dc_red) / (ac_ir / dc_ir)
    cal = self.skin_calibration_data[self.skin_combo.currentIndex()]
    spo2 = 104.0 - 17.0 * r + cal["spo2_offset"]
    return max(75.0, min(100.0, spo2))
```

3.13 Máquina de Estados de Movimiento, Detección de Contacto y Jerarquía de Alertas (`MainWindow.update_frequency`)

Implementa la jerarquía de seguridad clínica en tiempo real: 1. **Batería Baja (< 3.4V)**: Suspende el cálculo de presión arterial. 2. **Sin Contacto (< 20,000 unidades en IR)**: Limpia displays a "--". 3. **Movimiento Fuerte ($|x_{acc}| > 0.35g$)**: Muestra alerta "*Mantén la muñeca quieta*" y retiene los últimos valores calculados. 4. **Movimiento Leve ($0.08g - 0.35g$)**: Cancela ruido mediante NLMS. 5. **Reposo ($< 0.08g$)**: Operación normal.

Código

```
def update_frequency(self):
    if self.battery_low:
        self.alert_banner.setText("⚠ BATERÍA BAJA (<3.4V) - CARGUE EL DISPOSITIVO (Estimación en pausa)")
        self.bp_value_label.setText("PAUSA")
        return

    if avg_raw_ir < 20000:
        self.alert_banner.setText("⚠ SIN CONTACTO DE PIEL - COLOQUE EL SENSOR EN LA MUÑECA")
        return

    if self.motion_state == "MOVIMIENTO_FUERTE":
        self.alert_banner.setText("⚠ MANTÉN LA MUÑECA QUIETA - ARTEFACTO DE MOVIMIENTO DETECTADO")
        if self.last_valid_sbp: self.bp_value_label.setText(f"{self.last_valid_sbp} / {self.last_valid_dbp}*")
        return
```

3.14 Renderizado Dinámico en Tiempo Real en Matplotlib con Señales

NLMS (`MainWindow.update_plot`)

Actualiza el osciloscopio virtual a 30 FPS mediante vectorización con `set_data`, graficando la señal limpia post-NLMS en verde fluorescente (`#00FFCC`) o rojo guinda (`#9D2449`).

Código

```
def update_plot(self):
    if not self.is_previewing or len(self.time_buffer) == 0: return
    y_data = np.array(self.nlms_ir_buffer)
    t_data = np.array(self.time_buffer)
    t_max = t_data[-1]; t_min = max(0, t_max - 5)
    mask = (t_data >= t_min) & (t_data <= t_max)
    self.data_line.set_data(t_data[mask], y_data[mask])
    self.ax.set_xlim(t_min, t_max)
    self.canvas.draw_idle()
```

3.15 Corrección del Error en Botón de Grabación y Manejo de Búferes Clínicos (`MainWindow.start_recording`)

Resuelve el fallo de ejecución en tiempo de ejecución (`AttributeError: 'MainWindow' object has no attribute 'recording_raw'`), asegurando la inicialización correcta de todos los arreglos de registro para la validación clínica.

Código

```
def start_recording(self):
    if not self.is_previewing: return
    self.recording_time.clear()
    self.recording_raw_red.clear()
    self.recording_raw_ir.clear()
    self.recording_filtered_red.clear()
    self.recording_filtered_ir.clear()
    self.recording_nlms_red.clear()
    self.recording_nlms_ir.clear()
    self.recording_accel_x.clear()
    self.recording_accel_y.clear()
    self.recording_accel_z.clear()
    self.recording_accel_mag.clear()
    self.is_recording = True
```

3.16 Exportación CSV Extendida (N=30) con Banderas y Validación

Estadística (`MainWindow.save_csv`)

Genera un archivo CSV estructurado que incluye en sus primeras líneas el bloque de metadatos de configuración algorítmica y columnas para aceleración física, señales post-NLMS y referencias clínicas obligatorias (`ID_Sujeto` , `SBP_Referencia` , `DBP_Referencia`).

Código

```
def save_csv(self):
    with open(filename, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["# REPORTE DE VALIDACIÓN CLÍNICA - TENSIÓMETRO DIGITAL v2.0"])
        writer.writerow([f"# ID_Sujeto: {self.subject_id}", f"SBP_Ref: {self.sbp_reference:.1f}", f"DBP_Ref: {self.dbp_reference:.1f}"])
        writer.writerow(["Tiempo_s", "PPG_Rojo_Cruda", "PPG_IR_Cruda", "PPG_Rojo_NLMS", "PPG_IR_NLMS", "Accel_X_g", "Accel_Y_g", "Accel_Z_g", "BPM", "SpO2", "SBP", "DBP", "ID_Sujeto"])
```

3.17 Punto de Entrada Principal con Bucle Asíncrono `qasync` (`main`)

Punto de entrada de la aplicación. Integra el bucle asíncrono de eventos `asyncio` con el hilo principal de la interfaz gráfica de `PyQt6` mediante `QEventLoop`.

Código

```
if __name__ == "__main__":
    app = QApplication(sys.argv)
    loop = QEventLoop(app)
    asyncio.set_event_loop(loop)
    window = MainWindow()
    window.showMaximized()
    with loop:
        loop.run_forever()
```

4. Módulos de la Aplicación Web (HTML5 / CSS3 / ES6+) - Versión 2.0

4.1 Estructura Semántica del Monitor Web y Badges de Telemetría

([landing/index.html](#))

Define la estructura DOM del monitor médico web responsivo, incorporando badges para nivel de batería, contacto dérmico y estado de movimiento.

Código

```
<div class="status-indicator-box">
  <span id="connectionStatus" class="status-badge status-disconnected">● Desconectado</span>
  <span id="batteryStatus" class="status-badge status-battery">🔋 Batería: --%</span>
  <span id="skinContact" class="status-badge status-warning">⚠ Sin contacto</span>
  <span id="motionStatus" class="status-badge status-motion">🏃 Reposo</span>
</div>
<div id="alertBanner" class="alert-banner alert-normal">● SISTEMA EN ESPERA</div>
```

4.2 Modal de Calibración de Esfigmomanómetro en la Web

([landing/index.html](#))

Formulario modal incrustado para capturar las 3 mediciones previas de referencia médica del usuario.

Código

```
<div id="calibModal" class="modal-backdrop" style="display: none;">
  <div class="modal-content">
    <h3>🩺 Calibración Clínica Individual</h3>
    <input type="text" id="calibSubjectId" value="SUJETO-01">
    <input type="number" id="calibS1" value="120">
    <input type="number" id="calibD1" value="80">
    <button id="saveCalibBtn" class="btn btn-primary">Guardar y Aplicar</button>
  </div>
</div>
```

4.3 Estilizado Visual Hospitalario, Badges de Energía y Banner de Alerta

([landing/style.css](#))

Hojas de estilo con identidad clínica mexicana, animaciones de pulso y diseño responsivo para dispositivos móviles y de escritorio.

Código

```
.status-battery {  
    background-color: rgba(40, 167, 69, 0.15);  
    color: #28a745;  
    border: 1px solid rgba(40, 167, 69, 0.3);  
}  
.alert-banner {  
    width: 100%;  
    margin: 8px 0;  
    padding: 8px 10px;  
    border-radius: 6px;  
    font-weight: 700;  
    text-align: center;  
}  
.alert-battery {  
    background-color: #FFDDDD;  
    color: #9D2449;  
    border: 1px solid #9D2449;  
    animation: status-pulse 1.0s infinite;  
}
```

4.4 Filtro Digital Butterworth SOS en JavaScript ([landing/index.js](#))

Réplica exacta en ES6+ del filtro digital IIR de 4.º orden en cascada SOS.

Código

```
class ButterworthFilter {
  constructor() {
    this.sos = [
      [1.78260999e-03, 3.56521998e-03, 1.78260999e-03, -1.29176642e+00, 4.35717576e-01],
      [1.00000000e+00, 2.00000000e+00, 1.00000000e+00, -1.51260719e+00, 7.19524086e-01],
      [1.00000000e+00, -2.00000000e+00, 1.00000000e+00, -1.93751823e+00, 9.38713070e-01],
      [1.00000000e+00, -2.00000000e+00, 1.00000000e+00, -1.97736736e+00, 9.78372608e-01]
    ];
    this.reset();
  }
  filter(x) {
    let val = x;
    for (let i = 0; i < 4; i++) {
      const b = this.sos[i];
      const w = val - b[3] * this.states[i][0] - b[4] * this.states[i][1];
      const y = b[0] * w + b[1] * this.states[i][0] + b[2] * this.states[i][1];
      this.states[i] = [w, this.states[i][0]];
      val = y;
    }
    return val;
  }
}
```

4.5 Filtro Adaptativo NLMS para Cancelación de Ruido en JavaScript

([landing/index.js](#))

Implementación nativa del filtro NLMS en el navegador.

Código

```
class NLMSFilter {
  constructor(numTaps = 24, mu = 0.02, epsilon = 1e-5) {
    this.numTaps = numTaps;
    this.mu = mu;
    this.epsilon = epsilon;
    this.reset();
  }
  filter(desired, noiseRef, adapt = true) {
    for (let i = this.numTaps - 1; i > 0; i--) this.buffer[i] = this.buffer[i - 1];
    this.buffer[0] = noiseRef;
    let noiseEst = 0;
    for (let i = 0; i < this.numTaps; i++) noiseEst += this.weights[i] * this.buffer[i];
    const cleanSignal = desired - noiseEst;
    if (adapt) {
      let power = this.epsilon;
      for (let i = 0; i < this.numTaps; i++) power += this.buffer[i] * this.buffer[i];
      const step = (this.mu / power) * cleanSignal;
      for (let i = 0; i < this.numTaps; i++) this.weights[i] += step * this.buffer[i];
    }
    return cleanSignal;
  }
}
```

4.6 Conexión GATT y Servicio Web Bluetooth con Característica de Batería

([landing/index.js](#))

Establece la conexión GATT desde navegadores basados en Chromium utilizando la API Web Bluetooth.

Código

```
async function connectToDevice() {  
  bleDevice = await navigator.bluetooth.requestDevice({  
    filters: [{ namePrefix: 'Tensiometro_' }],  
    optionalServices: [SERVICE_UUID, "battery_service"]  
  });  
  bleServer = await bleDevice.gatt.connect();  
  bleService = await bleServer.getPrimaryService(SERVICE_UUID);  
  dataChar = await bleService.getCharacteristic(DATA_CHAR_UUID);  
  await dataChar.startNotifications();  
  dataChar.addEventListener('characteristicvaluechanged', onNotificationReceived);  
}
```

4.7 Decodificación de Trama de 20 Bytes y Aceleración Física Triaxial

([landing/index.js](#))

Decodifica las muestras binarias de 3 bytes (uint24) y los 6 bytes de aceleración triaxial (int16 Little Endian).

Código

```
function onNotificationReceived(event) {  
    const view = event.target.value;  
    if (view.byteLength < 14) return;  
    const red1 = (view.getUint8(0) << 16) | (view.getUint8(1) << 8) | view.getUint8(2);  
    const ir1 = (view.getUint8(3) << 16) | (view.getUint8(4) << 8) | view.getUint8(5);  
    const seq = view.getUint16(12, true);  
  
    let ax = 0, ay = 0, az = 1.0;  
    if (view.byteLength >= 20) {  
        ax = view.getInt16(14, true) / 16384.0;  
        ay = view.getInt16(16, true) / 16384.0;  
        az = view.getInt16(18, true) / 16384.0;  
    }  
    processIncomingSamples([red1], [ir1], seq, ax, ay, az);  
}
```

4.8 Motor de Detección Automática de Tono de Piel por Razón DCraw en Web

Adapta automáticamente el fototipo melánico en el cliente web mediante el cociente continuo $\$DC_{\{Rojo\}} / DC_{\{IR\}}\$$.

Código

```
if (irRaw > 20000 && currentMotionState === "REPOSO") {  
  dcRedAvg = 0.998 * dcRedAvg + 0.002 * redRaw;  
  dcIrAvg = 0.998 * dcIrAvg + 0.002 * irRaw;  
  stableContactCount++;  
  if (stableContactCount >= 500 && autoSkinCheckbox.checked) {  
    const ratio = dcRedAvg / dcIrAvg;  
    let autoIdx = 2;  
    if (ratio > 1.25) autoIdx = 0;  
    else if (ratio > 1.10) autoIdx = 1;  
    else if (ratio > 0.95) autoIdx = 2;  
    else if (ratio > 0.80) autoIdx = 3;  
    else if (ratio > 0.65) autoIdx = 4;  
    else autoIdx = 5;  
    skinToneSelect.value = autoIdx.toString();  
  }  
}
```

4.9 Corrección de la Indexación del Array en el Selector de Piel del DOM

([landing/index.js](#))

Solución técnica al error donde `parseInt(skinToneSelect.value) || 2` convertía erróneamente el índice `0` ("Muy clara") en `2`.

Código

```
function getSelectedSkinIndex() {  
  if (!skinToneSelect) return 2;  
  const val = parseInt(skinToneSelect.value, 10);  
  return isNaN(val) ? 2 : val; // Preserva exactamente el índice 0  
}
```

4.10 Modelo Hemodinámico PWA Web con Offset Clínico Individual

([landing/index.js](#))

Calcula los valores sistólicos y diastólicos con calibración personalizada en el navegador.

Código

```
function estimateBloodPressure(bpm, peaks, smoothed) {  
  const skinIdx = getSelectedSkinIndex();  
  const cal = skinCalibrationData[skinIdx];  
  let sbp = 120.0 + 0.15 * (bpm - 70.0) - 75.0 * (avgRise - 0.12) + cal.sbpOffset;  
  let dbp = 80.0 + 0.08 * (bpm - 70.0) - 25.0 * (avgFall - 0.35) + cal.dbpOffset;  
  if (isClinicallyCalibrated) {  
    sbp += calibSbpOffset;  
    dbp += calibDbpOffset;  
  }  
  return { sbp: Math.round(sbp), dbp: Math.round(dbp) };  
}
```

4.11 Bucle Gráfico de Animación a 60 FPS con Persistencia de Brillo

(`landing/index.js`)

Ejecuta el barrido continuo del osciloscopio en el elemento `<canvas>` utilizando `requestAnimationFrame`.

Código

```
function animationLoop() {  
  renderPlot();  
  requestAnimationFrame(animationLoop);  
}  
animationLoop();
```

4.12 Exportador y Generador de Dataset CSV Extendido en el Navegador

([landing/index.js](#))

Construye el archivo CSV extendido con metadatos y dispara su descarga directa en el cliente.

Código

```
function exportToCSV() {
    let csvContent = "data:text/csv;charset=utf-8,";
    csvContent += "# REPORTE CLINICO TENSIOMETRO DIGITAL v2.0 (ARCADIA)\n";
    csvContent += `Tiempo_s,PPG_Rojo_Cruda,PPG_IR_Cruda,PPG_IR_Limpia_NLMS,Accel_X_g,Accel_Y_g,Accel_Z_g,BPM,SpO2,SBP,DBP,ID_Sujeto\n`;
    for (let i = 0; i < recTime.length; i++) {
        csvContent += `${recTime[i].toFixed(4)},${recRawRed[i]},${recRawIr[i]},${recNlmsIr[i].toFixed(2)},${recAccelX[i].toFixed(4)},${recAccelY[i].toFixed(4)},${recAccelZ[i].toFixed(4)},${recBpm[i].toFixed(1)},${recSpo2[i].toFixed(1)},${recSbp[i]},${recDbp[i]},${subjectId}\n`;
    }
    const link = document.createElement("a");
    link.setAttribute("href", encodeURI(csvContent));
    link.setAttribute("download", `estudio_${subjectId}.csv`);
    document.body.appendChild(link);
    link.click();
}
```

5. Módulos de Empaquetado, Sincronización y Validación Clínica

5.1 Especificación PyInstaller para Generación del Ejecutable Autónomo Windows (`tensiometro-final.spec`)

Compila la aplicación PyQt6 en un binario autónomo `.exe` de 64 bits para Windows que opera sin necesidad de entorno Python ni dependencias externas.

Código

```
# -*- mode: python ; coding: utf-8 -*-
a = Analysis(
    ['tensiometro_reconstructed.py'],
    pathex=[],
    binaries=[],
    datas=[],
    hiddenimports=['bleak', 'qasync', 'matplotlib', 'numpy', 'PyQt6'],
    hookspath=[],
    runtime_hooks=[],
    excludes=[],
    noarchive=False,
)
pyz = PYZ(a.pure)
exe = EXE(
    pyz,
    a.scripts,
    a.binaries,
    a.datas,
    [],
    name='tensiometro-final',
    debug=False,
    bootloader_ignore_signals=False,
    strip=False,
    upx=True,
    console=False,
    icon=None
)
```

5.2 Automatización de Control de Versiones con Descarte de Datasets

Pesados (`push.bat` y `.gitignore`)

Automatiza la sincronización silenciosa con el repositorio GitHub descartando carpetas de datos masivos (> 3.3 GB) para respetar las cuotas de GitHub.

Código

```
@echo off
git add .
git commit -m "Actualizacion formal del sistema v2.0 con filtrado NLMS y monitoreo de bateria"
git push -u origin main
```

5.3 Protocolo de Validación Estadística Clínica (RMSE < 5 mmHg, Bland-Altman e ICC > 0.85)

Script de validación clínica automatizada que ingesta los datasets generados (\$N=30\$ sesiones de pacientes) y calcula los indicadores de validación biomédica:

Código

```
import numpy as np
import scipy.stats as stats

def evaluate_clinical_metrics(sbp_estimated, sbp_reference):
    differences = np.array(sbp_estimated) - np.array(sbp_reference)
    rmse = np.sqrt(np.mean(differences ** 2))
    mean_bias = np.mean(differences)
    std_diff = np.std(differences)

    # Concordancia de Bland-Altman (Límites de acuerdo al 95%)
    loa_upper = mean_bias + 1.96 * std_diff
    loa_lower = mean_bias - 1.96 * std_diff

    # Coeficiente de Correlación Intraclass (ICC 2,1)
    r, p_val = stats.pearsonr(sbp_estimated, sbp_reference)

    print(f"--- REPORTE DE VALIDACIÓN CLÍNICA (N={len(sbp_estimated)}) ---")
    print(f"• Error Cuadrático Medio (RMSE): {rmse:.2f} mmHg (Criterio: < 5.0 mmHg)")
    print(f"• Sesgo Medio: {mean_bias:.2f} mmHg")
    print(f"• Límites de Acuerdo Bland-Altman: [{loa_lower:.2f}, {loa_upper:.2f}] mmHg")
    print(f"• Coeficiente de Correlación (r): {r:.3f} (Criterio: > 0.85)")
    return rmse < 5.0 and r > 0.85
```